

PORTADA

Técnicas de Programación

Unidad 0 — Paradigma Imperativo: Sintaxis y Operadores

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

**Una instrucción a la vez.
En el orden exacto en que se escribió.**



CONCEPTO

El paradigma imperativo

Un **paradigma de programación** es una forma de pensar y organizar la solución a un problema en código — no es un lenguaje, es un enfoque que varios lenguajes pueden compartir.

El **paradigma imperativo** describe un programa como una **secuencia de instrucciones que modifican el estado** del programa paso a paso: se asigna, se compara, se repite, se decide.

Java, C, C# y Python (en su uso más común) siguen este paradigma — es el que usaremos como base durante todo este curso.



CONCEPTO

Sintaxis, semántica y pragmática

Perspectiva	Qué estudia	Ejemplo
Sintaxis	Las reglas para escribir una instrucción válida	<code>int x = 5;</code> es válido; <code>int 5 = x;</code> no lo es
Semántica	El significado de esa instrucción	<code>int x = 5;</code> significa "reservar memoria para un entero y guardar 5"
Pragmática	Cómo el contexto cambia la interpretación del problema	El mismo problema se resuelve distinto según el lenguaje y las herramientas disponibles

Como en cualquier idioma: las letras forman palabras, las palabras forman oraciones. En un lenguaje de programación, los caracteres forman sentencias, y un conjunto de sentencias forma las instrucciones del programa.

CONTRAEJEMPLO

Un mismo significado, dos sintaxis

```
// Java
System.out.println("Hola Mundo");

# Python
print("Hola Mundo")
```

Ambas instrucciones tienen la **misma semántica** — imprimir un texto en consola — pero una **sintaxis completamente distinta**. Confundir "cómo se escribe" con "qué significa" es un error común al empezar.

CONCEPTO · REPASO

Variables y literales

Una **variable** es un espacio de memoria cuyo valor puede cambiar durante la ejecución del programa; un **literal** es la representación directa de un valor en el código fuente (`5` , `"Hola"` , `true`).

```
int edad = 20;    // edad es variable, 20 es literal
edad = 21;        // el valor cambia, la variable sigue siendo la misma
```

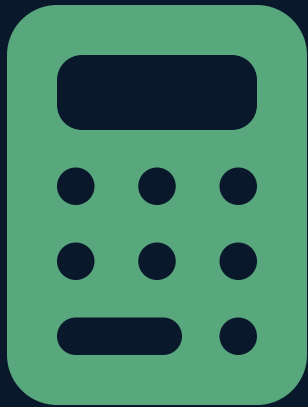
Los operadores que siguen trabajan sobre estos dos elementos: variables y literales.

CONCEPTO

Operadores aritméticos en Java

Operador	Significado	Ejemplo
+	Suma	a + b
-	Resta	a - b
*	Multiplicación	a * b
/	División	a / b
%	Módulo (residuo de la división)	a % b

Ojo con la división entera: `7 / 2` en Java da `3` , no `3.5` , porque ambos operandos son `int` . Para obtener el decimal, al menos uno debe ser `double` .



CONCEPTO

Jerarquía y precedencia

Igual que en álgebra, Java evalúa una expresión aritmética siguiendo un **orden de precedencia**: primero paréntesis, luego multiplicación/división/módulo, y por último suma/resta.

```
int resultado = 2 + 3 * 4;    // 14, no 20  
int resultado2 = (2 + 3) * 4; // 20
```

Cuando la precedencia no es obvia para quien lee el código, usar paréntesis explícitos es más claro que confiar en que todos recuerden el orden.

CONCEPTO

Operadores relacionales

Operador	Significado
<	Menor que
<=	Menor o igual que
>	Mayor que
>=	Mayor o igual que
==	Igual a
!=	Distinto de

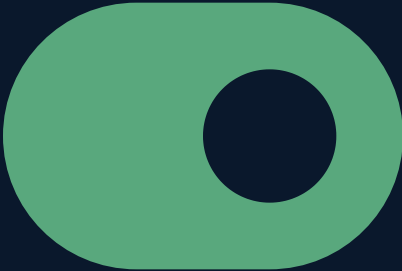
Todos devuelven un valor `boolean` (`true` o `false`) — **son la base de cualquier condición** que se evalúe más adelante.

CONCEPTO

Operadores lógicos: Y, O, NO

Operador	Nombre	Se cumple cuando...
&&	Y (AND)	ambas condiciones son verdaderas
	O (OR)	al menos una condición es verdadera
!	NO (NOT)	invierte el valor de la condición

```
boolean puedeVotar = edad >= 18 && tieneCedula;  
boolean feriado = esDomingo || esFestivo;  
boolean estaCerrado = !estaAbierto;
```



INTERACCIÓN

¿Verdadero o falso?

Con `a = 5` , `b = 10` y `c = 15` , evalúen en papel (sin ejecutar) el resultado de estas expresiones:

1. `a + b * 2 > c`
2. `(a < b) && (b < c)`
3. `!(a == 5) || (c > 20)`

5 minutos · comparen su respuesta con la persona de al lado antes de correr el código

SÍNTESIS

Lo que hay que llevarse de hoy

1. El paradigma imperativo describe un programa como una secuencia de instrucciones que modifican el estado.
2. Sintaxis es la forma; semántica es el significado; pragmática es el contexto de uso.
3. Los operadores aritméticos siguen una jerarquía de precedencia — los paréntesis nunca sobran cuando hay duda.
4. Los operadores relacionales siempre devuelven `boolean` ; los operadores lógicos (`&&` , `||` , `!`) combinan esos booleanos.



CIERRE

Antes de la próxima sesión

- Practicar 3-4 expresiones aritméticas y lógicas por cuenta propia, verificando el resultado en el IDE.
- Repasar la sintaxis de `if / else` en cualquier lenguaje que conozcan — se formaliza en Java en la próxima clase.

La próxima sesión toma estos operadores y los usa para construir condicionales y ciclos: las estructuras de control del paradigma imperativo.

